

HANDLING MISSING VALUES AND INAPPROPRIATE DATA IN A DATA SET

AIM

To demonstrate an experiment to handle missing data and inappropriate data in a data set using the Python pandas library for data preprocessing.

PROGRAM

```
import numpy as np
import pandas as pd

# Upload Hotel.csv and convert it into a DataFrame
df = pd.read_csv("Hotel_Dataset.csv")
print(df)

# Identify the duplicate row (row 9)
print(df.duplicated())
df.info()

# Remove duplicate rows
df.drop_duplicates(inplace=True)
print(df)
print("Number of rows:", len(df))

# Reset the index
index = np.array(list(range(0, len(df))))
df.set_index(index, inplace=True)
print(df)

# Drop the repeated column (axis=1 -> column)
df.drop(['Age_Group.1'], axis=1, inplace=True)

# Inappropriate values are replaced by NaN
df['CustomerID'] = df['CustomerID'].where(df['CustomerID'] >= 0)
df['Bill'] = df['Bill'].where(df['Bill'] >= 0)
df['EstimatedSalary'] = df['EstimatedSalary'].where(df['EstimatedSalary'] >= 0)
df['NoOfPax'] = df['NoOfPax'].where(df['NoOfPax'].between(1, 20))
df['Rating(1-5)'] = df['Rating(1-5)'].where(df['Rating(1-5)'].between(1, 5))
print(df)
print(df.isnull().sum())

# Inconsistent categorical values
print(df.Age_Group.unique())
print(df.Hotel.unique())
df['Hotel'] = df['Hotel'].replace('Ibys', 'Ibis')
print(df.Hotel.unique())
df['FoodPreference'] = df['FoodPreference'].replace(
    {'Vegetarian': 'Veg', 'veg': 'Veg', 'non-Veg': 'Non-Veg'})

# Fill the missing values
# numerical continuous -> mean, numerical discrete -> median, categorical -> mode
df['EstimatedSalary'] = df['EstimatedSalary'].fillna(round(df.EstimatedSalary.mean()))
df['NoOfPax'] = df['NoOfPax'].fillna(round(df.NoOfPax.median()))
df['Rating(1-5)'] = df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()))
df['Bill'] = df['Bill'].fillna(round(df.Bill.mean()))
print(df)
print(df.isnull().sum())
```

OUTPUT

```
CustomerID Age_Group Rating(1-5) Hotel FoodPreference Bill NoOfPax EstimatedSalary Age_Group.1
0 1 20-25 4 Ibis veg 1300 2 40000 20-25
1 2 30-35 5 LemonTree Non-Veg 2000 3 59000 30-35
2 3 25-30 6 RedFox Veg 1322 2 30000 25-30
3 4 20-25 -1 LemonTree Veg 1234 2 120000 20-25
4 5 35+ 3 Ibis Vegetarian 989 2 45000 35+
5 6 35+ 3 Ibys Non-Veg 1909 2 122220 35+
6 7 35+ 4 RedFox Vegetarian 1000 -1 21122 35+
7 8 20-25 7 LemonTree Veg 2999 -10 345673 20-25
8 9 25-30 2 Ibis Non-Veg 3456 3 -99999 25-30
9 9 25-30 2 Ibis Non-Veg 3456 3 -99999 25-30
10 10 30-35 5 RedFox non-Veg -6755 4 87777 30-35
0 False
1 False
2 False
3 False
4 False
5 False
6 False
7 False
8 False
9 True
10 False
dtype: bool
<class 'pandas.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
# Column Non-Null Count Dtype
---
0 CustomerID 11 non-null int64
1 Age_Group 11 non-null str
2 Rating(1-5) 11 non-null int64
3 Hotel 11 non-null str
4 FoodPreference 11 non-null str
5 Bill 11 non-null int64
6 NoOfPax 11 non-null int64
7 EstimatedSalary 11 non-null int64
8 Age_Group.1 11 non-null str
dtypes: int64(5), str(4)
memory usage: 924.0 bytes
CustomerID Age_Group Rating(1-5) Hotel FoodPreference Bill NoOfPax EstimatedSalary Age_Group.1
0 1 20-25 4 Ibis veg 1300 2 40000 20-25
1 2 30-35 5 LemonTree Non-Veg 2000 3 59000 30-35
2 3 25-30 6 RedFox Veg 1322 2 30000 25-30
3 4 20-25 -1 LemonTree Veg 1234 2 120000 20-25
4 5 35+ 3 Ibis Vegetarian 989 2 45000 35+
5 6 35+ 3 Ibys Non-Veg 1909 2 122220 35+
6 7 35+ 4 RedFox Vegetarian 1000 -1 21122 35+
7 8 20-25 7 LemonTree Veg 2999 -10 345673 20-25
8 9 25-30 2 Ibis Non-Veg 3456 3 -99999 25-30
10 10 30-35 5 RedFox non-Veg -6755 4 87777 30-35
Number of rows: 10
CustomerID Age_Group Rating(1-5) Hotel FoodPreference Bill NoOfPax EstimatedSalary Age_Group.1
0 1 20-25 4 Ibis veg 1300 2 40000 20-25
1 2 30-35 5 LemonTree Non-Veg 2000 3 59000 30-35
2 3 25-30 6 RedFox Veg 1322 2 30000 25-30
3 4 20-25 -1 LemonTree Veg 1234 2 120000 20-25
4 5 35+ 3 Ibis Vegetarian 989 2 45000 35+
5 6 35+ 3 Ibys Non-Veg 1909 2 122220 35+
6 7 35+ 4 RedFox Vegetarian 1000 -1 21122 35+
7 8 20-25 7 LemonTree Veg 2999 -10 345673 20-25
8 9 25-30 2 Ibis Non-Veg 3456 3 -99999 25-30
9 10 30-35 5 RedFox non-Veg -6755 4 87777 30-35
CustomerID Age_Group Rating(1-5) Hotel FoodPreference Bill NoOfPax EstimatedSalary
0 1 20-25 4.0 Ibis veg 1300.0 2.0 40000.0
1 2 30-35 5.0 LemonTree Non-Veg 2000.0 3.0 59000.0
2 3 25-30 NaN RedFox Veg 1322.0 2.0 30000.0
3 4 20-25 NaN LemonTree Veg 1234.0 2.0 120000.0
4 5 35+ 3.0 Ibis Vegetarian 989.0 2.0 45000.0
5 6 35+ 3.0 Ibys Non-Veg 1909.0 2.0 122220.0
6 7 35+ 4.0 RedFox Vegetarian 1000.0 NaN 21122.0
7 8 20-25 NaN LemonTree Veg 2999.0 NaN 345673.0
8 9 25-30 2.0 Ibis Non-Veg 3456.0 3.0 NaN
9 10 30-35 5.0 RedFox non-Veg NaN 4.0 87777.0
CustomerID 0
Age_Group 0
Rating(1-5) 3
Hotel 0
FoodPreference 0
Bill 1
NoOfPax 2
EstimatedSalary 1
dtype: int64
<StringArray>
['20-25', '30-35', '25-30', '35+']
Length: 4, dtype: str
```

```

<StringArray>
['Ibis', 'LemonTree', 'RedFox', 'Ibys']
Length: 4, dtype: str
<StringArray>
['veg', 'Non-Veg', 'Veg', 'Vegetarian', 'non-Veg']
Length: 5, dtype: str

```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1	20-25	4.0	Ibis	Veg	1300.0	2.0	40000.0
1	2	30-35	5.0	LemonTree	Non-Veg	2000.0	3.0	59000.0
2	3	25-30	4.0	RedFox	Veg	1322.0	2.0	30000.0
3	4	20-25	4.0	LemonTree	Veg	1234.0	2.0	120000.0
4	5	35+	3.0	Ibis	Veg	989.0	2.0	45000.0
5	6	35+	3.0	Ibis	Non-Veg	1909.0	2.0	122220.0
6	7	35+	4.0	RedFox	Veg	1000.0	2.0	21122.0
7	8	20-25	4.0	LemonTree	Veg	2999.0	2.0	345673.0
8	9	25-30	2.0	Ibis	Non-Veg	3456.0	3.0	96755.0
9	10	30-35	5.0	RedFox	Non-Veg	1801.0	4.0	87777.0
	CustomerID	0						
	Age_Group	0						
	Rating(1-5)	0						
	Hotel	0						
	FoodPreference	0						
	Bill	0						
	NoOfPax	0						
	EstimatedSalary	0						
	dtype:	int64						

RESULT

Thus the duplicate rows, inappropriate values, inconsistent categories and missing values in the Hotel data set were handled using the pandas library.